

Starting 7:05

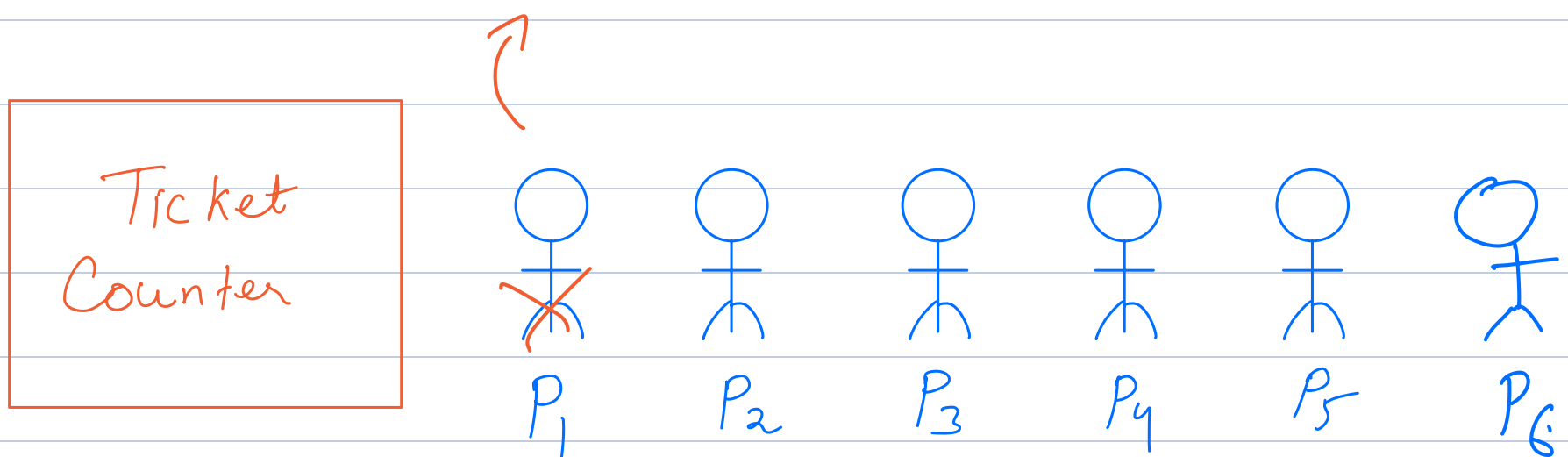
Today's Agenda:

1. Introduction To Queues
2. Implementation using Array
3. Implementation using Stacks
4. Doubly Ended Queue
5. Sliding Window Maximum ~~AA~~.

Queue: A linear data structure that follows the **FIFO** discipline

↑
First In First Out

Ex:-



Some more examples:-

1. Printer
2. Super Market Billing Queue
3. Call Centre calls
4. Message Queue (SAS, Kafka.)

Operations on Queue

① Enqueue (x) :- Insert an element at the end of the queue.

② Dequeue (x) :- Removes an element from the front of the queue.

③ Front() / Peek() :- Tells which element is there at the front of the queue.

④ isEmpty() :- Tells whether the queue is empty or not.

Quiz 1:

Eq(3) ✓	dq() ✓
Eq(7) ✓	Eq(8) ✓
Eq(12) ✓	Eq(3) ✓
dq() ✓	

3	7	12	8	3
--------------	--------------	----	---	---

Queue

12, 8, 3

Quiz 2

Eq(4) / Eq(7) /
dq() / Eq(11) /
Eq(9) / Eq(20) /
Eq(3) / dq() /

~~4~~ ~~9~~ 3 7 11 20

Queue

3, 7, 11, 20

Array Implementation of Queues

$(f+1) \cdot N$

front $\rightarrow$ idx of the last element dequeued

rear $\rightarrow$ idx of the last element enqueued

Size = ~~0~~, ~~2~~, ~~2~~, ~~1~~, ~~2~~, ~~2~~

$(r+1) \cdot N$

Eq(3) / dq() /
Eq(7) / Eq(8) /
Eq(12) / Eq(3) /
dq() / front() = 12

eq(5) /
eq(-1) $\rightarrow$ dq() - 1

5	-1	12	3	3
0	1	2	3	4

5

Also maintain a size variable to keep track of all the elements present in the queue.

void enqueue(int x) {

int dequeue() {

if (Size == N) {

if (Size == 0) {

Queue is full.

Throw error;

Queue is empty;

throw error;

rear = (rear + 1) % N;

arr[rear] = x;

Size++;

front = (front + 1) % N;

Size--;

```
int front() {
```

```
if (size == 0) {
```

```
    // Queue is empty;  
    // throw error;  
}
```

```
return arr[(front+1) % N]
```

```
}
```

```
bool isEmpty() {
```

```
return (size == 0);
```

```
}
```

Stack Implementation of Queue. (Doe-I)

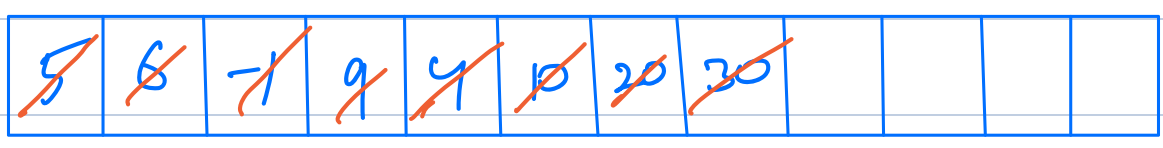
Implement

- a) Enqueue(x)
 - b) Dequeue()
 - c) Front()
 - d) isEmpty()
- $> \underline{\text{Amortised } O(1)}$

Using

push(x); pop(); top(); isEmpty();

Queue



eq(5) dq() - ⑤

eq(6) eq(10)

eq(-1) eq(20)

eq(9) eq(30)

eq(4) dq() ①

dq() - ① dq() - ③

dq() - ① dq() - ④

dq() - ① dq() - ①

st₁st₂

In our case, first 5 dq. operations

1 → 5

2 → 1

3 → 1

Avg: of first 5

$4 \rightarrow 1$
 $5 \rightarrow 1$

$$\frac{5+4}{5} = \underline{\underline{1.8}}$$

Amortised cost.

Generalising,

Enqueue operation always takes constant time since we are always pushing element to st1.

For a dequeue;

If st2 is empty:-

Push all elements from st1 to st2 & remove from top of st2

else:-

Remove from top of Stack2

If my st1 has x elements & st2 has 0 elements.

T.C. of DQ

$1^{st} \rightarrow x$
 $2^{nd} \text{ to } x^{th} \rightarrow \underline{\underline{1}}$

$$\therefore \text{Avg of first } x \text{ dq operations} = \frac{x + 1 \times (x-1)}{x}$$

$$= \frac{2x-1}{x} = 2 - \frac{1}{x}$$

Code

Stack <int> s1, s2;

$\approx O(1)$
Amortised
Cost

```
void enqueue(int x) {  
    s1.push(x);  
}
```

```
bool isEmpty() {  
    return (s1.isEmpty() &&  
            s2.isEmpty());  
}
```

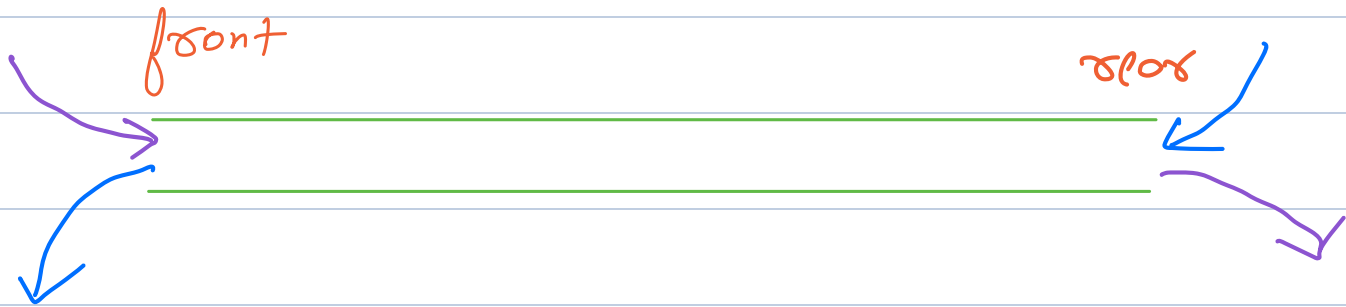
```
int front() {  
    if (isEmpty()) {  
        throw error;  
    }  
    if (s2.isEmpty()) {  
        while (!s1.isEmpty()) {  
            x = s1.top();  
            s1.pop(); s2.push(x);  
        }  
        return s2.top();  
    } else {  
        return s2.top();  
    }  
}
```

```
int dequeue() {  
    if (isEmpty()) {  
        throw error;  
    }  
    if (s2.isEmpty()) {  
        while (!s1.isEmpty()) {  
            x = s1.top();  
            s1.pop(); s2.push(x);  
        }  
        s2.pop();  
    } else {  
        s2.pop();  
    }  
}
```

8:25

Deque :- Doubly Ended Queue

Both the ends
data can be inserted & removed.



addLast(); addFront()
removeLast(); removeFront()
getFront(); getLast()
isEmpty();

Sliding Window Maximum

→ Find max of every window of size k .

arr → { 3, 2, 3, 4, 5, 5, 4, 5, 6 } $k=4$

_{0 1 2 3 4 5 6 7 8}

Windows

0 → 3 4

1 → 4 5

2 → 5 5

3 → 6 5

4 → 7 5

5 → 8 6

ans: { 4, 5, 5, 5, 5, 6 }

Quiz 3

_{0 1 2 3 4}
{ 1, 4, 3, 2, 5 }

$k=3$

Window

0 — 2 4

1 — 3 4

2 — 4 5

ans: { 4, 4, 5 }

Brute Force :- Iterate on every window of size k to find the max. Store it in the ans.

$$(N-k+1) \times k ; k = N/2$$

$$T.C \rightarrow O(N^2)$$

Applying Sliding Window

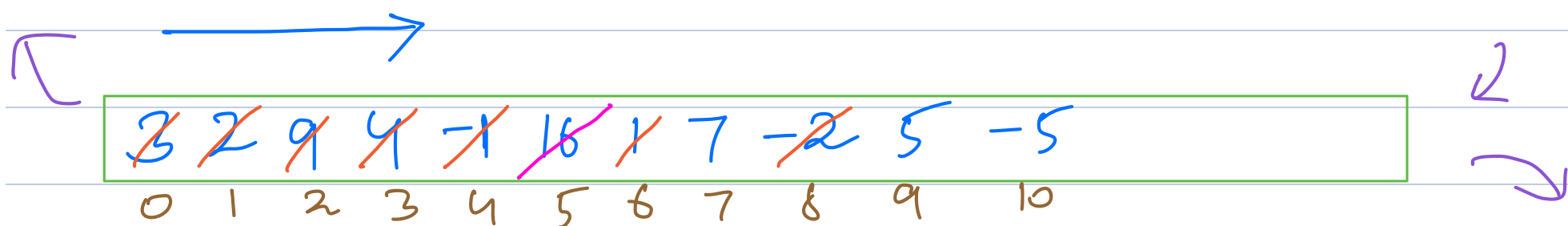
{ 3, 2, 9, 4, -1, 16, 1, 7, -2, 5, -5 }.

Simple Sliding Window is not working.
 $k=4$

Optimisation

$$\text{front} > i-k$$

{ 3, 2, 9, 4, -1, 16, 1, 7, -2, 5, -5 }.



Double Ended Queue

At any point, an element that cannot be largest of a subarray will be evicted.

1. Remove all smaller elements from the box.

2. Insert into the box my current element.

3. Update the ans. Carefully.

↓ check if

front > i-k

ans: { 9, 9, 16, 16, 16, 16, 7, 73 }

Code.

Deque <int> dq; list <int> ans;

for (i = 0; i < k; i++) {

 while (!dq.isEmpty() &&

 arr[dq.getLast()] ≤ arr[i]) {

 dq.removeLast();

 dq.addLast(i);

 ans.add(arr[dq.getFirst()]);

for (i = k; i < N; i++) {

 while (!dq.isEmpty() &&

 arr[dq.getLast()] ≤ arr[i]) {

 dq.removeLast();

 dq.addLast(i);

 if (dq.getFirst() ≤ i-k) {

 dq.removeFirst();

ans.add(arr[dq.front()]);

return ans;

TC $\rightarrow O(N)$

SC $\rightarrow O(K)$